Assignment

Source Control with Git, GitHub, and Visual Studio

GenAI Level 1 — Limited AI Use ⚠️

This assignment is classified as **Level 1 — Limited AI Use**. Please follow the **Level 1 GenAI requirements** described in the *Generative AI Use in Assessments* policy posted at the top of the Assignments section in Moodle.

Scenario — Community Food Bank Inventory Tracker

A local community food bank is developing a small C# application to keep track of donated food items. The organization wants the development team to use proper **source control practices** so that changes to the application can be tracked, reviewed, recovered, and safely integrated.

You have joined the project as a junior developer. Your task is to create a simple **Food Bank Inventory Tracker** in C# and, more importantly, manage its development using **Git, GitHub, and Visual Studio**.

The primary focus of this assignment is **not the complexity of the C# application**. The application serves as a practical project through which you will demonstrate the Git workflow covered in class: working directory → review/add → commit → local repository → push → remote repository, together with branching, merging, diff, fetch/pull, and version history.

---

1. Learning Objectives

After completing this assignment, you should be able to:

- explain the purpose of source control;
- distinguish between **Git and GitHub**;
- distinguish between a **working directory, local repository, and remote repository**;
- create and publish a Git repository using Visual Studio;
- review and stage changes;
- create meaningful commits;
- push commits to GitHub;

- inspect file differences using a diff;
- create and work on a branch;
- merge completed work into the main branch;
- use fetch and pull appropriately;
- inspect repository and commit history;
- demonstrate how Git can recover from an unwanted change;
- explain the difference between a commit and a push.

---

2. Part A — Create the Initial C# Application

Using Visual Studio, create a new **C# Console App (.NET)** project named:

`FoodBankInventory`

The initial application should contain a simple menu or output displaying information about donated food items.

Each item should contain at least:

- `Item ID`
- `Item Name`
- `Category`
- `Quantity`

Create at least **five sample food items**.

Your initial program should display the inventory in a readable format.

For example:

=== COMMUNITY FOOD BANK INVENTORY ===

ID: 101
Item: Canned Beans
Category: Canned Food
Quantity: 24

ID: 102
Item: Rice
Category: Dry Food
Quantity: 15

Keep the application relatively simple. The main purpose is to provide a realistic project whose evolution can be managed using Git.

---

3. Part B — Create the Git Repository

Add the Visual Studio project to **Git source control**.

Using Visual Studio:

1. initialize/connect the project to Git;
2. connect your GitHub account if required;
3. create/publish the remote repository on GitHub;
4. make the repository **private** unless instructed otherwise;
5. verify that the project files appear in the repository.

Your project should now have both:

Local Git Repository
↕
Remote GitHub Repository

Note
Remember that Git and GitHub are not the same thing: **Git is the version-control system, while GitHub hosts remote Git repositories.**

---

4. Part C — Make Your Initial Commit

Review the files that will be included in the repository.

Create an initial commit with a meaningful message, such as:

```
Create initial food bank inventory application
```

Then push the commit to GitHub.

Verify on GitHub that the commit and project files are available remotely.

Important

You should understand the distinction:

`Commit → Local Repository`

`Push → Remote Repository`

Note
A **commit does not automatically mean that the change has been uploaded to GitHub**. The material specifically distinguishes committing locally from pushing the resulting commits to the remote repository.

---

5. Part D — Modify, Review, Commit, and Push

Modify your application by adding a new piece of functionality.

For example, add the ability to display:

`Total number of inventory units`

Before committing:

1. save the modified file;
2. open the Git Changes/source-control view;
3. identify the modified file;
4. inspect the **diff**;
5. verify that the displayed changes are intentional;
6. stage/select the appropriate changes;
7. enter a meaningful commit message;
8. commit the changes;
9. push the commit to GitHub.

A suitable `commit` message might be:

`Add total inventory calculation`

Avoid meaningless `commit` messages such as:

- `changes`
- `stuff`
- `update`
- `commit`

Note
`commit` messages should allow another developer to understand what changed without inspecting every modified file.

---

6. Part E — Demonstrate a `Diff`

Make another small but meaningful modification.

For example, change the application's heading from:

```
Food Bank Inventory
```

to:

```
Community Food Bank Inventory System
```

Before committing, use Visual Studio to inspect the **difference between the previous and current versions**.

Take a screenshot. Your screenshot should clearly demonstrate which content was removed and which content was added.

Briefly explain in your report:

- what was changed;
- how Visual Studio represents the difference;
- why reviewing a diff before committing is useful.

---

7. Part F — Create and Use a Feature Branch

Your organization now requests a new feature:

Display all food items whose quantity is below a specified threshold so staff can identify items that need restocking.

Do **not** implement this feature directly on the main branch.

Create a new branch with an appropriate name, for example:

```
feature/low-stock-report
```

Then:

1. verify that you are working on the new branch;
2. implement the low-stock functionality;
3. compile and run the application;
4. verify that the feature works correctly;
5. review the changes;
6. create a meaningful commit;
7. push the branch to the remote repository.

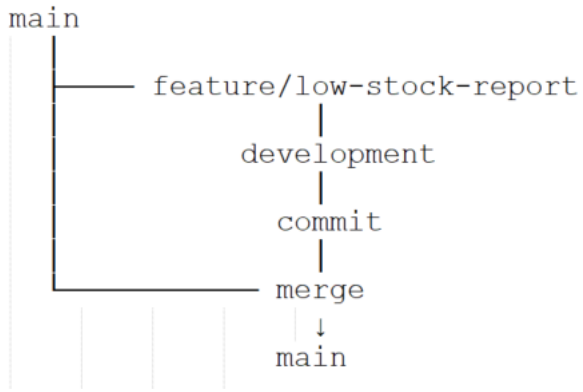
Note
The purpose of the branch is to isolate feature development from the stable main version, which is one of the central branching concepts in the provided material.

---

8. Part G — Merge the Feature

After testing the low-stock feature, integrate it into the main branch.

Your workflow should demonstrate:



After the merge:

1. build the complete application;
2. run it;
3. verify that the original functionality still works;
4. verify that the new low-stock feature works;
5. push the updated main branch to GitHub.

Note
A successful Git merge does not guarantee that the application is correct. Git combines changes, but the resulting application must still be built and tested.

---

9. Part H — Demonstrate Version History and Recovery

Make a temporary, intentional change that you do **not** want to keep.

For example, temporarily change:

```
Community Food Bank Inventory System
```

to:

```
TEST VERSION
```

Use Git/Visual Studio's version-control functionality to inspect the change and restore the intended version.

Your report should document:

1. the original version;
2. the unwanted change;
3. how Git identified the modification;
4. how you restored the intended code;
5. the final corrected application.

The purpose is to demonstrate why source control is more than simply keeping backup copies of files: Git maintains the history and evolution of the project.

---

10. Part I — Fetch and Pull

Demonstrate that you understand the purpose of **Fetch** and **Pull**.

Use the Git/Visual Studio interface to perform the appropriate operations and document them.

In your report, briefly explain the difference:

`Fetch:` `Check/retrieve information about remote changes`

`Pull:` `Retrieve and integrate remote changes`

If you are completing the assignment individually and there are no new remote changes available, you may still demonstrate the operations and explain what Visual Studio reports.

---

11. Part J — Repository and Commit History

Inspect your repository history.

By the end of the assignment, your repository should contain **multiple meaningful commits** that demonstrate the evolution of your application.

For example:

```
Create initial food bank inventory application
```

```
Add total inventory calculation

Improve application heading

Implement low-stock report

Merge low-stock feature
```

Note
Do not create all functionality first and then make artificial commits afterward. Your commit history should demonstrate that the application was developed **incrementally**.

---

12. Part K — Git Concepts in Your Report

Include a short section titled:

Git Concepts Demonstrated

Using your own project as the example, briefly explain the difference between:

- Git and GitHub;
- working directory and repository;
- local repository and remote repository;
- commit and push;
- fetch and pull;
- branch and merge;
- clone and fork;
- pull and pull request.

You do not necessarily need to perform every operation, such as forking or creating a pull request, unless instructed. However, you should be able to explain the distinction based on the course material.

---

13. Testing Requirements

Your final application must compile and run successfully.

At minimum, demonstrate:

- the five initial food items;
- inventory information displayed correctly;

- total inventory calculation;
- low-stock functionality;
- successful execution before and after the feature merge;
- successful execution after restoring the unwanted change.

Do not assume that a successful Git operation means the application works. Build and test the project after significant changes and after merging.

---

14. Report and Screenshot Requirements

Submit a **short but complete report documenting the practical work performed during the assignment**.

The report must include a **comprehensive set of screenshots** demonstrating that you personally completed the required Git and Visual Studio activities.

Your screenshots should clearly show:

1. the initial Visual Studio project and working application;
2. the project being added to Git/source control;
3. the GitHub remote repository;
4. the initial commit;
5. a modified file appearing in Git Changes;
6. a meaningful **diff**;
7. a meaningful commit message;
8. a successful push to GitHub;
9. repository/commit history;
10. creation of the feature branch;
11. development occurring on the feature branch;
12. the feature branch commit;
13. the merge into main;
14. successful execution after the merge;
15. the temporary unwanted modification;
16. restoration/recovery of the intended version;
17. fetch/pull operations;
18. the final repository on GitHub;
19. the final Visual Studio project;
20. the final console output demonstrating the required functionality.

Each major screenshot must include a **short caption or explanation describing what it demonstrates**.

Important — Screenshots Are Part of the Practical Assessment

Showing only the final C# source code and the final GitHub repository is **not sufficient**.

The screenshots must demonstrate the **development and source-control process**, including changes, diffs, commits, pushes, branching, merging, recovery, and testing.

---

15. Submission Requirements

Submit the following through **Moodle**:

1. Your complete Visual Studio C# project, including all source code.
2. Your report containing the comprehensive set of screenshots demonstrating the implementation, Git workflow, execution, and testing of the practical requirements.

All submitted files must be clearly named and organized.

You must respect the due date specified on Moodle.

Also include the **required GenAI Disclosure Statement** according to the Level 1 policy posted on Moodle.

---

16. Mandatory Presentation to the Professor

In addition to submitting the assignment on Moodle, you must **present and demonstrate your completed work to the professor to receive a grade**.

During the presentation, you may be asked to:

- run the application;
- show the Git repository;
- show the remote GitHub repository;
- explain the difference between commit and push;
- show your commit history;
- explain a diff;
- switch between branches;
- explain why the feature was developed on a separate branch;

- explain or demonstrate a merge;
- explain fetch versus pull;
- demonstrate how Git identifies a modified file;
- explain how you recovered from the unwanted change;
- make a small change, commit it, and explain the workflow.

Important Grading Requirement

Submission on Moodle alone is not sufficient. The assignment must also be presented to the professor to receive a grade. An assignment that is submitted but not presented will not be graded.

---

17. Assessment Rubric

Criterion	Weight	Expectations
Git Repository & Basic Workflow	25%	Correct repository setup and meaningful use of add/stage, commit, push, diff, and GitHub
Branching, Merging & Version Management	25%	Correct feature branch workflow, merge, history inspection, and recovery of an unwanted change
C# Application & Testing	15%	Working application with required functionality; application built and tested throughout development
Git Understanding & Commit Quality	15%	Clear understanding of Git concepts and meaningful incremental commit history
Report & Practical Screenshots	20%	Clear, organized report with comprehensive screenshots demonstrating the required practical workflow
Total	**100%**	

Mandatory GenAI Disclosure Requirement

For this **Level 1 assessment**, the required **GenAI Disclosure Statement must be included in the submission**, whether GenAI was used or not.

Failure to include the required GenAI Disclosure Statement will result in a grade of zero (0) for the assignment.